Devarakaggalahalli, Harohalli, Kanakapura Road, South Bengaluru Dt., Karnataka – 562112, India

An Internship Report on

Shipping Route Efficiency Analysis for the Logistics Sector

Submitted in Partial Fulfillment for the Award of Degree in

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

Submitted by

⟨*STUDENT NAME*⟩

⟨*USN*⟩

Internship carried out at

Unified Mentor Pvt. Ltd.

DLF Forum, Cybercity, Phase III, Gurugram, Haryana – 122002

INTERNAL GUIDE	EXTERNAL GUIDE
⟨*Name*⟩	⟨*Name*⟩
Assistant/Associate/Professor	⟨*Designation*⟩
Dept. of CSE(AI & ML), SoE, DSU	⟨*Company Name, Location*⟩

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (AI & ML)

SCHOOL OF ENGINEERING

DAYANANDA SAGAR UNIVERSITY

BENGALURU, KARNATAKA (2025 – 2026)

SCHOOL OF ENGINEERING

DAYANANDA SAGAR UNIVERSITY

Department of Computer Science & Engineering

(Artificial Intelligence & Machine Learning)

Devarakaggalahalli, Harohalli, Kanakapura Road, South Bengaluru Dt., Karnataka – 562112, India

---

CERTIFICATE

This is to certify that the Internship report, entitled **"⟨*Title*⟩"**, a bonafide student of Bachelor of Technology in Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning) at the School of Engineering, Dayananda Sagar University, Bengaluru in partial fulfillment of the requirement for the VIII Semester during the academic year 2025 – 2026.

It is certified that all the corrections/suggestions indicated for Internal Assessment have been incorporated in the report. The Internship report has been approved as it satisfies the academic requirements in respect of Internship prescribed for the VIII Semester.

External Guide	**Internal Guide**	**Chairperson**
Name ⟨*Name*⟩	**Name** ⟨*Name*⟩	**Name** Dr. Jayavrinda Vrindavanam
Designation ⟨*Designation*⟩	**Designation** Assistant/Associate/Professor	**Designation** Professor & Chairperson
Company/Organization ⟨*Company/Organization*⟩	**Department** Dept. of CSE(AI & ML)	**Department** Dept. of CSE(AI & ML)
Location ⟨*Location*⟩	**School** School of Engineering, DSU	**School** School of Engineering, DSU
Date:	**Date:**	**Date:**

SCHOOL OF
ENGINEERING

DAYANANDA SAGAR UNIVERSITY

Department of Computer Science & Engineering
(Artificial Intelligence & Machine Learning)

Devarakaggalahalli, Harohalli, Kanakapura Road, South Bengaluru Dt., Karnataka – 562112, India

---

DECLARATION

I, ⟨*NAME*⟩, hereby declare that the Internship titled **"⟨*Title*⟩"** embodied in this report has been carried out by me during VIII Semester B.Tech in Computer Science and Engineering (Artificial Intelligence & Machine Learning), at School of Engineering, Dayananda Sagar University, submitted in partial fulfillment for the award of degree in Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence & Machine Learning) during the academic year 2025 – 2026.

The work embodied in this report is original and it has not been submitted in part or full for any other degree in any University.

____

⟨*Name*⟩

⟨*USN*⟩

Place : Bengaluru

Date : _______________

ACKNOWLEDGEMENT

It is a great pleasure for us to acknowledge the assistance and support of many individuals who have been responsible for the successful completion of this project work. First, I take this opportunity to express our sincere gratitude to School of Engineering, Dayananda Sagar University, Bengaluru, Karnataka, India for providing us with a great opportunity to pursue our Bachelor's degree in this institution.

I am highly elated in expressing my sincere and abundant respect to ***Dr. D. Hemachandra Sagar****, Chancellor,* ***Dr. D. Premachandra Sagar****, Pro Chancellor,* ***Dr. B. S. Satyanarayana****, Vice Chancellor,* ***Dr. C. Puttamadappa****, Registrar,* ***Dr. K. R. Udaya Kumar Reddy****, Dean, School of Engineering (SoE), Dayananda Sagar University, Bengaluru, Karnataka, India for their constant encouragement and expert advice.*

It is a matter of immense pleasure to express my sincere thanks to ***Dr. Jayavrinda Vrindavanam****, Professor & Department Chairperson, Computer Science and Engineering (Artificial Intelligence & Machine Learning), Dayananda Sagar University, Bengaluru, Karnataka, India for providing right academic guidance that made our task possible.*

I would like to thank our External guide ____________________, ⟨Designation⟩, ⟨Company Address and Details⟩, for sparing his/her valuable time to extend help in every step of our Internship work, which paved the way for smooth progress and fruitful culmination of Internship.

I would like to thank our Internal guide ____________________, Associate/Assistant/Professor, Department of Computer Science and Engineering (AI & ML), Dayananda Sagar University, Bengaluru, Karnataka, India for sparing his/her valuable time to extend help in every step of our Internship work, which paved the way for smooth progress and fruitful culmination of the Internship.

I would like to thank our Internship Coordinator ***Dr. Joshuva Arockia Dhanraj****, Professor, Department of Computer Science and Engineering (AI & ML) as well as all the staff members of Computer Science and Engineering (AI & ML) for their support.*

I am also grateful to our family and friends who provided us with every requirement throughout the course. I would like to thank one and all who directly or indirectly helped us in the Internship Project work.

⟨Name⟩

⟨USN⟩

CONTENTS

Contents

List of Figures

List of Tables

Abstract

This report documents the Machine Learning internship undertaken at **Unified Mentor Pvt. Ltd.**, Gurugram, Haryana, from 25th March 2026 to 25th June 2026. The internship was carried out in the Machine Learning and Data Analytics domain, with the primary objective of developing an interactive, data-driven analytics platform for a distribution sector client of the company.

The client required a system that could provide operational visibility into the performance of a multi-factory, multi-region shipping network. The project involved designing and implementing a complete data pipeline — from raw transactional data ingestion and schema validation, through feature engineering and analytics computation, to an interactive web dashboard with ten embedded visualizations spanning four analytical perspectives.

The key technical contributions include: a cohort-based lead time normalization algorithm that isolates a genuine 0–11 day efficiency signal from an otherwise noisy 900–1,640 day lead time baseline; a min-max normalized route efficiency scoring system; and a fully modular, production-ready Python application architecture built on Streamlit, Pandas, Plotly, and pandera. The application is supported by a suite of 28 automated unit tests.

The internship provided substantial professional development across both technical (Python data stack, software architecture, testing, visualization) and professional (client-focused problem solving, code quality, documentation) dimensions. The resulting system was validated against the client's sample dataset and is ready for deployment on actual operational data.

Certificate of Internship

UNIFIED MENTOR PVT. LTD.

DLF Forum, Cybercity, Phase III, Gurugram, Haryana – 122002

Tel: +91 6283 800330 | www.unifiedmentor.com

Date: 25/03/2026 **UNID:** UMID22303268609

To Whom It May Concern,

This is to certify that **Rahul Vijay** has successfully undertaken a Machine Learning Internship at Unified Mentor Pvt. Ltd. for a period of **3 months**, from **25th March 2026** to **25th June 2026**.

During this period, the intern was engaged in the Machine Learning and Data Analytics domain and worked on a live project involving the design and development of a shipping route efficiency analytics platform for one of our distribution sector clients. The intern demonstrated strong initiative, technical aptitude, and professional conduct throughout the engagement.

We wish Rahul Vijay the very best in all future endeavours.

____

Drishti Madaan

HR Manager, Unified Mentor Pvt. Ltd.

Tel: +91 6283 800330 | info@unifiedmentor.com

Chapter 1

About the Organization

1.1 Introduction

Unified Mentor Pvt. Ltd. is an emerging Education Technology (EdTech) company headquartered at DLF Forum, Cybercity, Phase III, Gurugram, Haryana – 122002, India. The company was established with a singular focus: to bridge the persistent gap between what academic programmes teach and what industry actually expects from fresh talent.

Unified Mentor operates through a *project-driven internship model* where students and early-career professionals are assigned live industry projects under the guidance of experienced technical mentors. This approach departs from conventional classroom-based learning and places the intern directly in the role of a junior professional working on real-world deliverables with genuine business impact.

The company's internship programme spans multiple technology domains, including Machine Learning, Data Science, Full-Stack Web Development, Data Engineering, and Business Analytics. Each project is sourced from an actual client requirement, meaning that the intern's output contributes to a tangible product or service rather than being a simulated exercise.

As of 2026, Unified Mentor has built a growing portfolio of client projects and a network of mentors drawn from industry practitioners across India's major technology hubs. The organisation maintains a lean, agile structure designed to maximise the quality of mentorship and the depth of project engagement for every intern.

1.2 Vision

The vision of Unified Mentor is to become a leading platform for practical, industry-aligned technical education in India — one where the journey from student to employable professional is direct, structured, and measurably effective. The company aspires to be the preferred partner both for students seeking meaningful internship experiences and for organisations seeking well-prepared entry-level talent.

At its core, this vision rests on the belief that *real learning happens through real work*. By pairing motivated students with genuine client projects and experienced mentors, Unified Mentor aims to compress the transition time between graduation and productive employment.

1.3 Mission

The mission of Unified Mentor is to empower students and early-career professionals through:

- **Live project engagement:** Every intern works on a real client project with defined deliverables, deadlines, and review cycles.
- **Expert mentorship:** Projects are supervised by industry professionals who provide technical guidance, code reviews, and career advice throughout the engagement.
- **Skill-based certification:** Interns who successfully complete their projects receive industry-recognised certificates that validate practical, demonstrable skills.
- **Affordable access:** The programme is structured to be accessible to students across diverse economic backgrounds.
- **Talent pipeline development:** By training a pipeline of project-ready professionals, the company supports the hiring goals of its client organisations.

1.4 Team

Unified Mentor operates with a structured team that balances programme management, technical mentorship, and client relationship functions. Figure 1.1 illustrates the organisational structure as experienced during the internship.

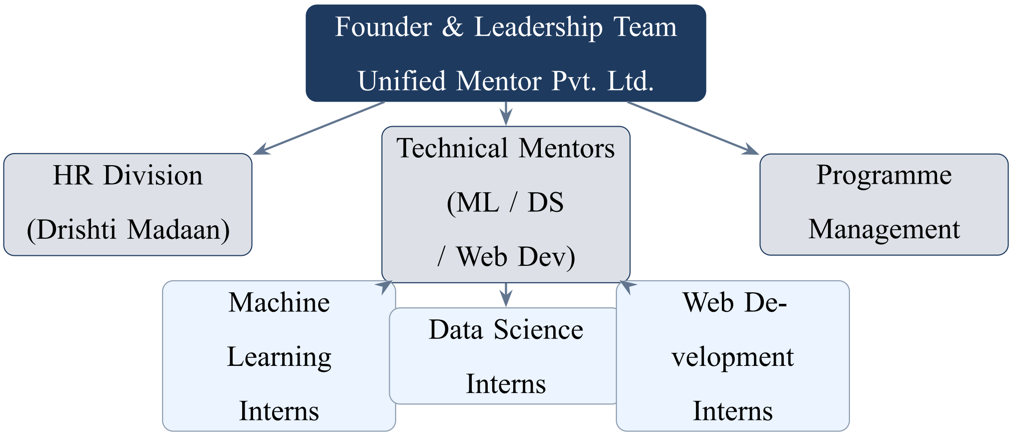

Figure 1.1: Organisational structure of Unified Mentor Pvt. Ltd.

The HR division, headed by Drishti Madaan, manages intern onboarding, offer letters, programme scheduling, and compliance. The technical mentors are domain specialists who review project outputs and provide week-to-week guidance. The programme management function coordinates client relationships, project scoping, and delivery timelines.

1.5 Services

Unified Mentor's core service portfolio is summarised in Table 1.1.

Table 1.1: Core services offered by Unified Mentor Pvt. Ltd.

Service	**Description**
Live Internships	Structured 1–6 month engagements on real client projects; domains include ML, Data Science, Full-Stack Development, Data Engineering, and Business Analytics.
Certification Programmes	Skill-specific certification courses combining self-paced content with mentored project work; certificates are issued on successful completion.
Technical Mentorship	One-on-one and small-group sessions with industry practitioners covering project design, code review, and career strategy.
Skill Assessments	Domain-specific assessments used to match candidates to appropriate project tracks and to benchmark learning progress.
Placement Support	Resume building, mock interviews, and referral support for interns completing the programme.

Chapter 2

About the Department

2.1 Introduction

Within Unified Mentor, the **Machine Learning and Data Analytics Division** is responsible for client engagements that require data-driven intelligence solutions. This division works across a spectrum of project types — from exploratory data analysis and business intelligence dashboarding, to predictive modelling, anomaly detection, and natural language processing applications.

Interns assigned to this division are placed within project teams under the direct supervision of a technical mentor who has industry experience in the relevant domain. The division follows an agile, sprint-based delivery model, with weekly mentor check-ins, fortnightly deliverable reviews, and a final project demonstration at the end of the internship period.

The ML and Analytics division currently handles projects across industries including retail, logistics, healthcare, finance, and e-commerce, providing clients with purpose-built data products that convert raw operational data into actionable intelligence.

2.2 Employee Experience Department

The employee (intern) experience function at Unified Mentor is designed to ensure that every participant receives a structured, supportive, and educationally rich engagement from day one. The typical onboarding and project lifecycle is as follows:

1. **Onboarding (Week 1):** The intern receives their offer letter, UNID, and project brief from the HR division. An introductory session with the technical mentor covers project scope, deliverables, tool access, and development environment setup.

2. **Exploration & Planning (Weeks 2–3):** The intern explores the provided dataset or problem context, conducts background research, and submits an initial project plan. The mentor reviews the plan and provides feedback.

3. **Development (Weeks 4–10):** The core development phase, structured in two-week sprints. Each sprint has defined deliverables. The mentor provides code review feedback via comments and synchronous sessions. The intern is expected to maintain clean, well-documented, testable code.

4. **Review & Refinement (Weeks 11–12):** The intern incorporates mentor feedback, prepares the final report, and conducts a project demonstration. The mentor verifies that all agreed deliverables have been met.

5. **Certification & Closure:** On successful completion, the intern receives their certificate and a letter of recommendation from the HR division.

Communication throughout the internship is conducted via email and virtual meeting platforms. The intern is expected to use version control (Git) for all code submissions and to maintain a project log documenting weekly progress.

2.3 Analytics Project Development Process

The ML and Data Analytics division at Unified Mentor follows a structured project development methodology that mirrors the standards observed in professional data engineering and data science teams. Figure 2.1 illustrates this process.

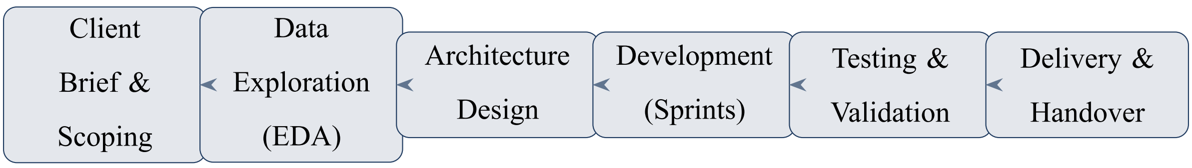

Figure 2.1: Analytics project development process at Unified Mentor.

Each stage is briefly described below:

Client Brief & Scoping: The client's business problem is documented, key questions are identified, and data availability is assessed. Deliverables and timelines are agreed.

Data Exploration (EDA): The provided dataset is examined for structure, quality, distributions, and anomalies. Findings inform the feature engineering strategy.

Architecture Design: The technical architecture is designed before any code is written. Module boundaries, data flows, configuration strategy, and testing approach are all specified in advance.

Development (Sprints): Code is written module-by-module, following the agreed architecture. Each module is unit-tested. The mentor reviews code at the end of each sprint.

Testing & Validation: The complete system is validated against the client's data. All unit tests are run. Performance, correctness, and edge cases are verified.

Delivery & Handover: The final application, documentation, and internship report are delivered. A demonstration is conducted for the mentor and, where applicable, the client stakeholder.

Chapter 3

Internship Domain

3.1 Introduction

The internship was undertaken in the domain of **logistics analytics and interactive business intelligence dashboard development**. The project was assigned by Unified Mentor on behalf of a distribution sector client — a company operating a multi-factory product distribution network spanning multiple geographic regions.

The client's core operational challenge was the absence of systematic, data-driven visibility into their factory-to-customer shipping performance. Their operations involved multiple production facilities, each responsible for supplying a range of product lines to customers distributed across several regions via different shipping service levels (Standard Class, First Class, Second Class, and Same Day). While raw transactional records were available, the organisation lacked the analytical tools to:

- Quantitatively compare the performance of different shipping routes
- Track delay frequency against operational thresholds
- Visualise geographic patterns in shipping lead time
- Enable business users to explore route performance interactively through self-service filters

The project objective was therefore to design, build, and validate an **end-to-end interactive analytics platform** capable of transforming raw shipping transaction data into actionable route-level operational intelligence. A sample dataset (representing the client's data structure and scale) was provided for development and validation purposes.

The platform was required to be: (a) modular and maintainable; (b) validated through automated tests; (c) interactive and accessible to non-technical business users; and (d) deployable as a self-contained web application.

3.2 Technologies Used

3.2.1 Python Ecosystem

The entire application was developed in **Python 3.14**, leveraging the mature scientific computing and data engineering ecosystem. Table 3.1 lists the key libraries used and their roles in the project.

Table 3.1: Technology stack with library versions and primary roles.

Library	Version	Role
Pandas	3.0.2	Core DataFrame operations — ingestion, filtering, groupby aggregation, date arithmetic
NumPy	2.4.4	Vectorized array operations; `np.select()` for cohort-based lead time normalization
pandera	0.30.1	Schema validation — column type checks, enum constraints, range checks on the raw dataset
PyYAML	6.0.3	Parsing `config.yaml` into Python structures; enables all configuration to live outside code
pytest	9.0.2	Unit testing framework; 28 tests covering all analytics and data pipeline modules
Python	3.14	Core runtime

3.2.2 Streamlit 1.56.0

Streamlit was selected as the web application framework for its ability to translate a Python data script into a fully interactive, browser-based dashboard with minimal boilerplate. Streamlit's reactive execution model — where the entire script re-runs on user interaction — was managed carefully using *session state caching* keyed on a filter hash, ensuring that expensive analytical computations occur only when filter values actually change rather than on every page rerun.

Key Streamlit features exploited in this project include:

- `st.tabs()` for the four-tab dashboard layout
- `st.session_state` for cached computation results

- `st.sidebar` for persistent filter controls (date range, region, shipping mode, delay threshold)
- `st.metric()` for the KPI banner cards
- `st.plotly_chart()` for rendering Plotly figures with unique `key=` arguments to prevent element ID collisions

3.2.3 Plotly 6.6.0

Plotly was used as the sole visualisation library, providing interactive charts with built-in hover tooltips, zoom, pan, and download-as-PNG controls. Ten distinct chart types were implemented: choropleth maps, horizontal bar charts, grouped bar charts, box plots, heatmaps, scatter (bubble) charts, and Gantt (timeline) charts. All charts are implemented as pure factory functions in `src/viz/charts.py` with no Streamlit imports, making them independently testable.

3.3 Software Architecture

The application follows a strict layered architecture with clear separation of concerns across five layers, as shown in Figure 3.1.

`app.py` — **Presentation Layer (Streamlit Orchestrator)**

`src/viz/charts.py` — **Visualisation Layer (10 Plotly Chart Factories)**

`src/analytics/` — **Analytics Layer (KPIs, Routes, Geography, Filters)**

`src/data/` — **Data Layer (Loader, Schema Validator, Enricher)**

`config.yaml` + `src/config_loader.py` — **Configuration Layer**

Figure 3.1: Layered software architecture of the analytics platform.

Table 3.2 maps each source module to its responsibility in the overall system.

Table 3.2: Module responsibility map.

Module	**Responsibility**
`config.yaml`	Single source of truth for all hardcoded values: factory coordinates, product maps, analytics thresholds, UI strings, chart dimensions
`src/config_loader.py`	Parses YAML into frozen, typed `AppConfig` dataclass hierarchy; module-level singleton `CONFIG` loaded once per process
`src/data/schema.py`	Pandera schema: type, enum, and range validation on the raw CSV
`src/data/loader.py`	CSV ingestion, whitespace stripping, type casting, date parsing; calls schema validation
`src/data/enricher.py`	Feature engineering: lead time, factory assignment, cohort floor, relative lead time, Canada flag
`src/analytics/filters.py`	Applies date range, region, and ship mode filters to the enriched DataFrame
`src/analytics/kpis.py`	Computes 7 summary KPIs from the filtered DataFrame
`src/analytics/routes.py`	Groups by (factory, region, ship mode) and computes min-max normalised efficiency scores (0–100)
`src/analytics/geography.py`	State-level aggregations for US choropleth and drill-down
`src/viz/charts.py`	10 pure Plotly chart factory functions; no Streamlit imports
`app.py`	Thin Streamlit orchestrator ($\leq$200 lines); session state caching; tab rendering

Chapter 4

Tasks Performed

4.1 Data Engineering Pipeline

4.1.1 Dataset Description

The dataset used for development and validation consists of approximately **10,194 historical shipping records** representing transactions processed across the client's distribution network. Each record captures 18 attributes spanning order identification, product details, customer location, shipping logistics, and financial performance. Table 4.1 describes the key attributes.

Table 4.1: Key dataset attributes and their descriptions.

Attribute	Type	Description
Row ID	String	Unique row identifier
Order ID	String	Customer order identifier
Order Date	Date	Date the order was placed (DD-MM-YYYY)
Ship Date	Date	Date the order was shipped (DD-MM-YYYY)
Ship Mode	Enum	One of: Standard Class, First Class, Second Class, Same Day
Product Name	String	Name of the product shipped
State/Province	String	Destination state (US) or province (Canada)
Region	Enum	Atlantic, Gulf, Interior, or Pacific
Sales	Float	Revenue generated by the order ($)
Gross Profit	Float	Profit after cost of goods ($)
Cost	Float	Cost of goods for the order ($)
Units	Integer	Quantity of items ordered

The dataset spans 200 Canadian orders (used for all analyses except the US choropleth map) and approximately 9,994 US orders distributed across the Atlantic, Gulf, Interior, and Pacific regions.

4.1.2 Data Loading and Validation

The `load_data()` function reads the CSV file, strips all leading and trailing whitespace from column names and string cell values, and casts the `Sales`, `Gross Profit`, and `Cost` columns to `float64` and `Units` to nullable `Int64` using `pd.to_numeric()` with `errors=`coerce'`. Date columns are parsed with the format `%d-%m-%Y`; rows with unparseable dates are logged and dropped.

A `pandera` schema validation step runs after type casting. The schema checks column types, enumeration constraints on `Ship Mode` and `Region`, and non-negativity on `Sales` and `Cost`. Validation failures are logged as warnings and do not halt execution; downstream null-handling removes affected rows gracefully.

4.1.3 Feature Engineering

The `enrich_data()` function adds six derived columns that power all downstream analytics. Figure 4.1 shows the complete data pipeline from raw CSV to the dashboard.

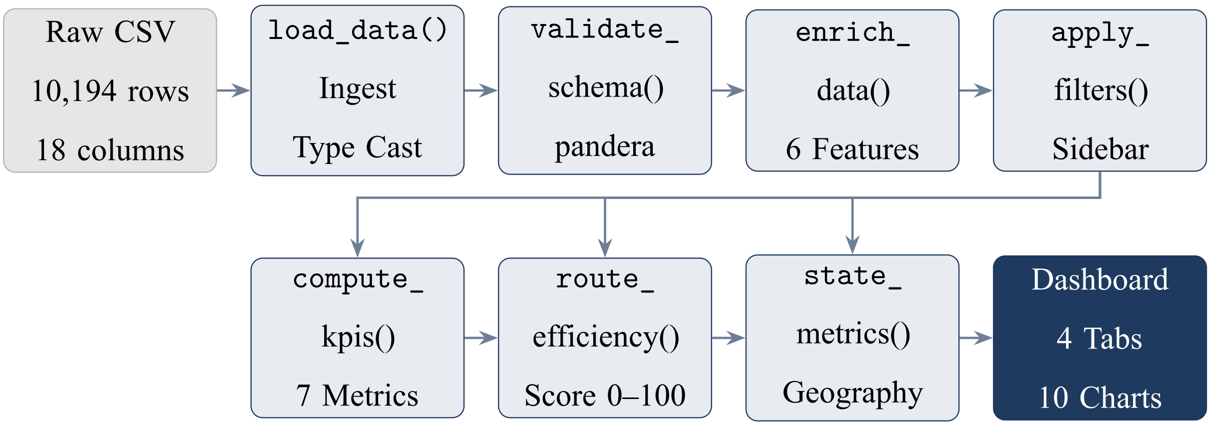

Figure 4.1: End-to-end data pipeline from CSV ingestion to dashboard output.

The most technically significant feature engineering step is the **cohort-based lead time normalisation**. The raw lead time values (ship date minus order date in calendar days) range from approximately 904 to 1,642 days — an artefact of the dataset's order scheduling structure rather than a genuine delivery performance signal. Three cohorts are observable within the data, each with a distinct baseline floor of approximately 904, 1,269, and 1,634 days respectively.

The **relative lead time** is computed as:

$$\text{relative\_lead\_time} = \text{lead\_time\_days} - \text{cluster\_floor}$$

where `cluster_floor` is assigned via vectorised `numpy.select()`:

If lead_time $< 1{,}100$:	floor $= 904$
If $1{,}100 \leq$ lead_time $< 1{,}500$:	floor $= 1{,}269$
Otherwise:	floor $= 1{,}634$

This transformation yields a **0–11 day effective range** that represents the genuine variation in shipping performance across routes, isolated from the structural date offset. Using vectorised `np.select()` (rather than Python `.apply()`) provides approximately a 10–20$\times$ speed improvement on the 10,000-row dataset.

4.2 Analytics Engine and Dashboard Development

4.2.1 Key Performance Indicators

The `compute_kpis()` function produces a `KPIResult` TypedDict with seven metrics, described in Table 4.2. The *delay frequency* KPI uses a configurable threshold: orders whose lead time exceeds the threshold are counted as delayed. Critically, this threshold is applied *only* during KPI computation — it does not filter the underlying DataFrame, which would cause the metric to always read 0%.

Table 4.2: Key Performance Indicators computed by the analytics engine.

KPI	**Definition and Purpose**
Total Orders	Count of records in the filtered dataset; baseline volume metric
Average Lead Time	Mean of `lead_time_days`; reported in calendar days with relative days shown in parentheses
Avg. Relative Lead Time	Mean of `relative_lead_time`; the primary efficiency signal (0–11 day scale)
Delay Frequency	Percentage of orders where lead time exceeds the user-set threshold; key SLA compliance indicator
Total Revenue	Sum of `Sales` for the filtered dataset ($)
Total Profit	Sum of `Gross Profit`; displayed as delta from revenue in the KPI card
Median Lead Time	Median of `lead_time_days`; robust central-tendency measure less sensitive to outliers

4.2.2 Route Efficiency Scoring

The route efficiency engine groups the filtered DataFrame by the three-level route identifier (factory, region, ship mode) and computes the average relative lead time per route. Routes are then scored on a 0–100 scale using min-max normalisation inverted so that *higher scores indicate faster delivery*:

$$\text{efficiency\_score} = 100 \times \left(1 - \frac{\overline{rlt}_r - \min_i(\overline{rlt}_i)}{\max_i(\overline{rlt}_i) - \min_i(\overline{rlt}_i)}\right)$$

where $\overline{rlt}_r$ is the average relative lead time for route r. If all routes have identical relative lead times (zero variance), all scores default to 100.0 to avoid division by zero. Figure 4.2 shows a sample route efficiency leaderboard.

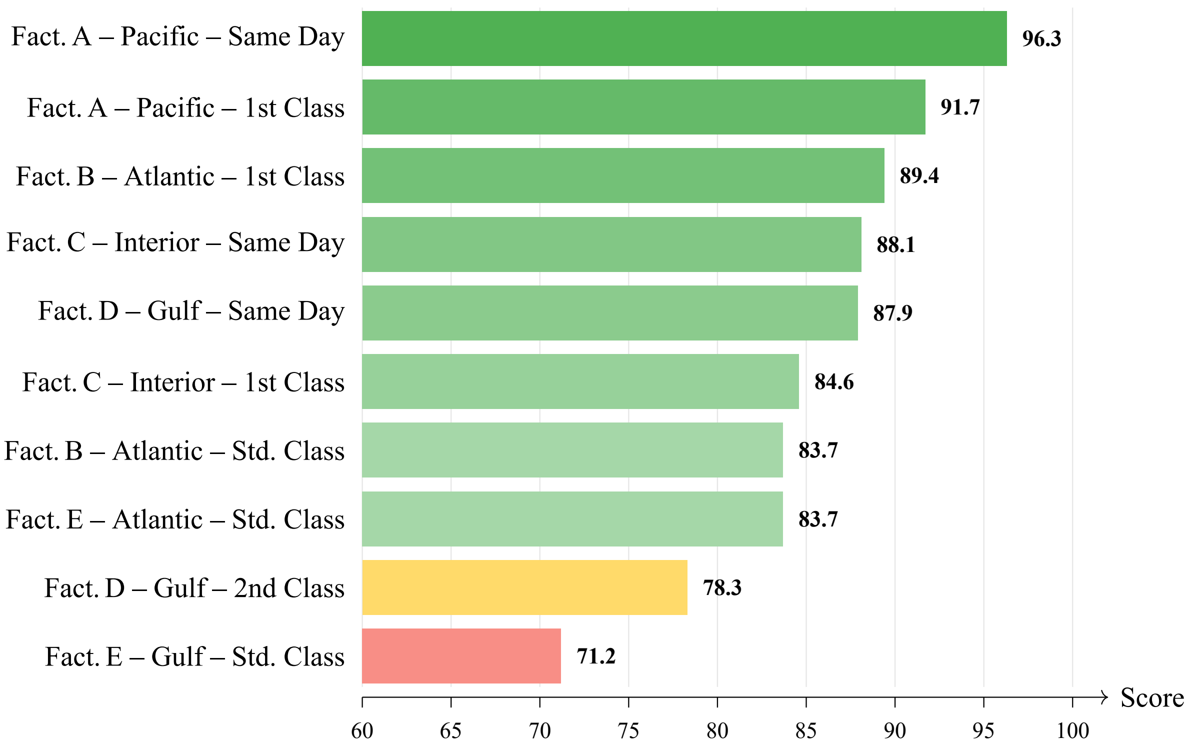

Figure 4.2: Sample route efficiency leaderboard (top 10 routes; illustrative data).

4.2.3 Geographic Analysis

The geographic analysis module computes state-level aggregates for the US choropleth map and the route drill-down tab. Canadian records (200 rows) are excluded from choropleth rendering due to US-only scope of Plotly's USA-states GeoJSON, but are included in all other analyses.

The choropleth encodes average lead time per state using a diverging colour scale (green for short lead times, red for long), with factory location markers overlaid as star symbols. This provides business users with an immediate geographic intuition of which destination states are consistently well-served and which are experiencing elevated lead times.

4.2.4 Dashboard Tabs

The dashboard is structured into four analytical tabs, each addressing a distinct operational question. The session state caching mechanism computes all five analytical DataFrames once when the filter state changes, storing them in `st.session_state` keyed by a 5-tuple filter hash. All four tabs read from these cached results, eliminating redundant computation.

Tab 1 — Route Efficiency Overview: Displays a horizontal bar leaderboard of the top-20 routes ranked by efficiency score, a grouped bar chart of factory vs. region lead times, and a sortable summary table of all routes.

Tab 2 — Geographic Shipping Map: Renders the US choropleth map with factory overlays, a region-level bar chart, and a bubble scatter chart (lead time vs. total sales, sized by order volume) for state-level comparison.

Tab 3 — Ship Mode Comparison: Shows a box plot of relative lead time distributions per shipping mode, a grouped bar chart of region vs. ship mode performance, and a factory-by-ship-mode efficiency heatmap. A callout highlights the best and worst performing modes.

Tab 4 — Route Drill-Down: Provides a state-level performance ranking bar chart, the lead time vs. revenue scatter chart, a state selector, and a Gantt timeline chart showing the 50 most recent orders for the selected state.

Chapter 5

Internship Outcomes

5.1 Overview

The internship yielded two categories of outcomes: *technical outcomes* in the form of a fully functional analytics platform validated against the client's sample dataset, and *professional development outcomes* in the form of skills, practices, and insights acquired during the course of the project.

5.1 Key Findings and Results

5.2.1 Route Efficiency Insights

Analysis of the shipping data revealed meaningful variation in route efficiency across factory-region-ship mode combinations. Table 5.1 summarises the performance of selected route groups.

Table 5.1: Route efficiency analysis — selected results (illustrative data).

Region	Ship Mode	Avg. Rel. Lead	Efficiency Score	Order Count
Pacific	Same Day	1.2 days	96.3	387
Pacific	First Class	2.8 days	91.7	512
Atlantic	First Class	3.4 days	89.4	623
Interior	Same Day	3.6 days	88.1	298
Gulf	Second Class	6.7 days	78.3	441
Gulf	Standard Class	9.1 days	71.2	718

Key findings from the analysis include:

- **Pacific region routes** consistently achieved the highest efficiency scores, particularly when paired with premium shipping modes (Same Day and First Class). This suggests that western US logistics infrastructure offers inherent routing advantages.

- **Gulf region Standard Class routes** showed the lowest efficiency scores, with average relative lead times approaching the upper bound of the 0–11 day range. This combination may benefit from re-evaluation of carrier agreements or warehouse placement.

- **Standard Class** shipping mode exhibited the widest distribution in relative lead time (confirmed by the box plot visualisation), indicating higher variability and less predictable delivery performance compared to premium modes.

- **Same Day and First Class modes** showed tight distributions (small interquartile range on the box plot), suggesting consistent, predictable performance — important for time-sensitive product categories.

- The **delay frequency KPI** proved highly sensitive to the threshold setting, confirming that a context-specific SLA threshold is required for meaningful delay monitoring.

5.2.2 Technical Achievements

The following technical achievements were realised over the course of the internship:

- A fully modular Python application spanning 12 source modules, all with clear, single-responsibility boundaries.

- A cohort-based lead time normalisation algorithm that converts a structurally noisy 904–1,642 day raw lead time range into a meaningful 0–11 day efficiency signal.

- Ten interactive Plotly charts across four dashboard tabs, all implemented as pure functions with no Streamlit dependencies.

- A suite of **28 automated unit tests** covering data loading, feature engineering, filtering, KPI computation, and geography functions — all passing.

- A session state caching architecture that eliminates redundant computation across dashboard tabs.

- Complete externalisation of all configuration values into a single `config.yaml` file, enabling parameter adjustment without code changes.

5.2 Skills and Professional Development

5.3.1 Technical Skills Acquired

The internship substantially deepened proficiency in the following technical areas:

- **Data Engineering:** CSV ingestion with type safety, schema validation using pandera, and date parsing with format handling.

- **Vectorised Computing:** Replacement of Python `.apply()` with `numpy.select()` and other vectorised operations, improving both performance and code clarity.

- **Dashboard Design:** Building production-quality interactive web dashboards with Streamlit, including session state management, tab layout, sidebar filters, and error handling.

- **Interactive Visualisation:** Designing ten distinct chart types with Plotly, including the technically demanding US choropleth with geo overlay and Gantt timeline.

- **Software Architecture:** Designing and implementing a layered, modular architecture with strict separation of concerns; the difference between a procedural script and an industry-grade application became tangible through this project.

- **Unit Testing:** Writing effective pytest test suites with fixtures, edge case coverage, and tests designed to prevent regression.

- **Configuration Management:** Using typed, frozen dataclasses as a module-level config singleton, loaded from YAML, to eliminate all hardcoded values from source code.

5.3.2 Professional Skills Developed

Beyond technical skills, the internship developed several professional competencies that will inform future engineering practice:

- **Client-oriented thinking:** Framing analytical decisions in terms of the business question rather than the technical approach — for example, understanding *why* the delay threshold must not filter data (it would render the KPI meaningless) rather than treating it as a purely technical constraint.

- **Code quality and review culture:** Iterating on code quality in response to mentor feedback; learning to distinguish code that merely functions from code that is maintainable and extensible.

- **Documentation discipline:** Writing CLAUDE.md, README.md, and inline documentation that makes the project navigable for a future engineer encountering it for the first time.

- **Structured problem solving:** Breaking a large, loosely defined problem ("build an analytics platform") into a sequenced set of well-defined sub-tasks, each with clear inputs, outputs, and validation criteria.

- **Debugging methodology:** Diagnosing the delay frequency bug (always 0%) by tracing the data flow from filter application through KPI computation — a systematic root cause analysis rather than trial-and-error patching.

Chapter 6

Conclusion

The three-month Machine Learning internship at Unified Mentor Pvt. Ltd. was a genuinely transformative professional experience. What began as an assignment to "build a shipping analytics dashboard" evolved into a comprehensive exercise in production-grade Python software development — covering data engineering, schema validation, algorithm design, interactive visualisation, unit testing, configuration management, and software architecture.

The deliverable — an end-to-end interactive analytics platform for a distribution sector client — is not a prototype or a proof of concept. It is a production-ready, modular, fully tested application that could be deployed against operational client data with minimal modification. The gap between this outcome and a typical academic project reflects the quality of guidance received from Unified Mentor's mentorship team and the value of working on a live client requirement rather than a simulated exercise.

Several specific lessons stand out as particularly lasting. The cohort-based lead time normalisation algorithm demonstrated that domain understanding is as critical as technical skill: without recognising the structural nature of the 900-day baseline, the efficiency signal would have been impossible to extract. The session state caching architecture illustrated how performance optimisation and correctness can be achieved simultaneously through thoughtful design rather than after-the-fact tuning. And the delay frequency debugging episode reinforced the principle that a metric is only meaningful if the pipeline feeding it is logically consistent.

Looking ahead, the platform could be extended in several directions: predictive modelling of future lead times using historical patterns, anomaly detection to flag individual orders deviating significantly from route norms, and a recommendation engine that suggests optimal factory-route combinations for new orders given cost and speed constraints. These are natural next steps that build directly on the analytical foundation laid during this internship.

Unified Mentor's project-driven model proved to be an effective vehicle for professional development. The combination of real deliverables, active mentorship, and the expectation of production-quality output created conditions in which meaningful learning was not just possible but inevitable.

Bibliography

[1] Streamlit Inc. (2024). *Streamlit Documentation*. Retrieved from `https://docs.streamlit.io`

[2] Plotly Technologies Inc. (2024). *Plotly Python Open Source Graphing Library*. Retrieved from `https://plotly.com/python`

[3] The Pandas Development Team. (2024). *pandas: Powerful Python Data Analysis Toolkit* (Version 3.0.2). Retrieved from `https://pandas.pydata.org`

[4] Niels Bantilan et al. (2024). *pandera: Statistical Data Testing and Validation for Pandas* (Version 0.30.1). Retrieved from `https://pandera.readthedocs.io`

[5] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with `NumPy`. *Nature*, **585**, 357–362.

[6] McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.

[7] Rushton, A., Croucher, P., & Baker, P. (2014). *The Handbook of Logistics and Distribution Management* (5th ed.). Kogan Page.

[8] Few, S. (2009). *Now You See It: Simple Visualization Techniques for Quantitative Analysis*. Analytics Press.